

Multi-core?

By default the Linux kernel supports upto 512 processor cores, you may want to tone that down a little

GNU?

Some Linux distributions call themselves GNU/Linux to imply that it is a GNU software distribution running on a Linux kernel

Workshop

and then enable / disable it as you find necessary.

Since we want to use a separate directory for our source and our build, we need to provide some additional commands over and above the ones we have listed above. We will add the following extra parameters to our make command:

```
make -C /usr/src/linux-2.6.36/ O=$PWD
```

The reason for this is simple. The “-C” option for make tells the make command to change to the specified directory (here “/usr/src/linux-2.6.36”) as before running the command. The O=\$PWD parameter sets the output directory to \$PWD, which is an environment variable that your shell sets as the current directory. This is a way of saying run the make command in that (/usr/src/linux-2.6.36/) directory but output the stuff here. These need to be added every time.

So go ahead and run:

```
make -C /usr/src/linux-2.6.36/ O=$PWD xconfig
```

OR

```
make -C /usr/src/linux-2.6.36/ O=$PWD gconfig
```

And configure the kernel according to your liking.

STEP 3: Building the kernel

Finally, we get to building the kernel. This is as simple as issuing the make command, or in our case:

```
make -C /usr/src/linux-2.6.36/ O=$PWD
```

To take advantage of that multi-core system you have, you can issue this command with the parameter -jN, where N is the number of parallel tasks you wish to perform. This can have a significant impact on the build time. In our case the build time on an Intel Core i5 2.67 GHz quad-core system with 8 GB RAM went from 50 minutes for a single thread build to 13 minutes with 8 threads. We recommend staying within 2x the number of cores in your processor. So for a quad-core system with 4 build in parallel:

```
make -C /usr/src/linux-2.6.36/ O=$PWD -j4
```

This step will take time, depending on



The Qt-based GUI for configuring the Linux kernel.

how fast your system is, how many cores your computer has and how many parallel tasks you enabled. This could take from a few minutes to over an hour.

STEP 4: Installing your new kernel

Till now all the commands we have run have been run as a normal user. However these final few steps need to be run as the super-user. So we assume you have switched to the super-user (using “su”) for the last few steps.

Now we need to install the modules for our kernel build:

```
make -C /usr/src/linux-2.6.36/
```



The Linux kernel compiling, one build at a time.

```
O=$PWD modules_install
```

Finally we install the kernel with:

```
make -C /usr/src/linux-2.6.36/
```

```
O=$PWD install
```

Now we may have one final task left, to create an initial ramdisk image “initrd” for your kernel. This will again depend on the distribution you are compiling for.

We need to go to the “/boot” directory and run mkinitrd.

Ubuntu and some other distributions use “mkinitramfs”, in this case run go to the /boot directory (still as a super-user or using “sudo”), and run:

```
update-initramfs -c -k  
vmlinuz-2.6.36
```

Finally, finally, finally you need to update your boot loader, this will depend on the distribution you are using. For newer versions of Ubuntu, this will be “update-grub2” otherwise “update-grub”. If your distribution uses some other boot-loader, please check the specific instructions for your boot loader (“lilo” can be updated using the “lilo” command).

It is a good idea to keep your old kernel around, at least until you are sure it is stable and all your hardware is working. You may also need to build any third partly kernel module such NVIDIA or ATI drivers for your new kernel.

Conclusion

Building you own Linux kernel has a plethora of advantages, and only some of them have been described in this article. While the information presented here is enough to get a build running, the rest of the burden is still on you. Read the “README” file which comes with the kernel sources, and refer to the extensive documentation available inside the Documentation folder included with the kernel sources for any specific configuration options. Finally, refer to your distribution's own guide to compiling the kernel as this may have additional features / options that the “vanilla” kernel might not.

Depending on how well versed with Linux you are, reading this article will either appease your concerns about compiling your own Linux kernel, or further turn you off the concept. If you find that the kernel is harder to build than you thought, you probably need to spend some more time using Linux before you are ready to take this step. Otherwise you are probably half-way through the build process already. In that case: Good Luck! 🍀